

SLIP 1

Q1. [10 Marks]

Create a Pandas DataFrame containing the names, ages and marks of five students . Display the first five records, dimensions of the DataFrame, column names and basic statistical summary.

```
import pandas as pd

# Create data
data = {
    "Name": ["Amit", "Priya", "Rahul", "Sneha", "Neha"],
    "Age": [20, 21, 20, 22, 21],
    "Marks": [85, 90, 78, 88, 92]
}

# Create DataFrame
df = pd.DataFrame(data)

# Display first five records
print("First Five Records:")
print(df.head())

# Display dimensions
print("\nDimensions of DataFrame:")
print(df.shape)

# Display column names
print("\nColumn Names:")
print(df.columns)

# Display basic statistical summary
print("\nStatistical Summary:")
print(df.describe())
```

Q2. [20 Marks]

Using the Diabetes dataset , use BMI as the independent variable and disease progression measure as the dependent variable.

1. Split the data into training and testing sets.
2. Build a Simple Linear Regression model.
3. Display the regression coefficient and intercept.

4. Calculate the residuals and display a scatter plot with the regression line.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Load the Diabetes dataset
df = pd.read_csv("diabetes.csv")
df.head()
df.columns

# Independent and dependent variables
X = df[["BMI"]]
y = df["Outcome"]
# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Create Simple Linear Regression model
model = LinearRegression()
# Train the model
model.fit(X_train, y_train)
# Display coefficient and intercept
print("Regression Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)
# Calculate residuals
residuals = y_test - y_pred
print("\nResiduals:")
print(residuals)
# Scatter plot and regression line
plt.scatter(X_test, y_test)
plt.plot(X_test, y_pred)
plt.xlabel("BMI")
plt.ylabel("Disease Progression")
plt.title("BMI vs Disease Progression")
plt.show()
```

Q3. Viva [5 Marks]

SLIP 2

Q1. [10 Marks]

Create a Pandas DataFrame containing student names and marks of five students . Display the students who scored more than 60 marks and sort the resulting records in descending order of marks.

```
import pandas as pd
# Create a DataFrame
data = {
    'Student Name': ['Aarav', 'Priya', 'Rahul', 'Sneha', 'Ananya'],
    'Marks': [75, 55, 88, 62, 48]
}
df = pd.DataFrame(data)
print( "Original DataFrame:")
print( df)
# Filter students who scored more than 60
result = df[df['Marks'] > 60]
# Sort in descending order of marks
result = result.sort_values(by='Marks', ascending=False)
print( "\nStudents scoring more than 60 marks:")
print( result)
```

Q2. [20 Marks]

Using the Cancer dataset , build a Logistic Regression model to classify observations as malignant or benign.

1. Split the dataset into training and testing sets.
2. Train the Logistic Regression model.
3. Calculate the accuracy score.
4. Display and interpret the confusion matrix.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
# Load dataset
df = pd.read_csv("cancer.csv")
# Remove unnecessary columns
df = df.drop(["id", "Unnamed: 32"], axis=1, errors="ignore")
```

```
# Separate features and target
X = df.drop("diagnosis", axis=1)
y = df["diagnosis"]
# Convert M = 1 and B = 0
y = y.map( {"M": 1, "B": 0})
# Split dataset
X_train, X_test, y_train, y_test = train_test_      split(
    X, y, test_size=0.2, random_state=42
)
# Create and train Logistic Regression model
model = LogisticRegression(max_iter=10000)
model.fit( X_train, y_train)
# Prediction
y_pred = model.predict( X_test)
# Accuracy
print( "Accuracy:", accuracy_score(y_test, y_pred))
# Confusion Matrix
print( "Confusion Matrix:")
print( confusion_matrix(y_test, y_pred))
```

Q3. Viva [5 Marks]

SLIP 3

Q1. [10 Marks]

Create a small Data Frame containing five students and their marks , **with at least two missing values. Identify the missing values and replace them using the** mean of the marks column .

```
import pandas as pd
```

```
# Create DataFrame with missing values
```

```
data = {  
    'Student Name': ['Aarav', 'Priya', 'Rahul', 'Sneha', 'Ananya'],  
    'Marks': [75, None, 88, None, 65]  
}
```

```
df = pd.DataFrame(data)
```

```
print( "Original DataFrame:")
```

```
print( df)
```

```
# Identify missing values
```

```

print( "\nMissing Values:")
print( df.isnull())

# Calculate mean of Marks column
mean_marks = df['Marks'].mean()
print( "\nMean of Marks:", mean_marks)

# Replace missing values with mean
df['Marks'] = df['Marks'].fillna(mean_marks)

print( "\nDataFrame after replacing missing values:")
print( df)

```

Q2. [20 Marks]

Consider the following advertising and sales data:

Advertising Expenditure	Sales
10	25
15	32
20	38
25	45
30	51
35	58
40	64
45	70
50	77

Build a Simple Linear Regression model using Advertising Expenditure as the independent variable and Sales as the dependent variable.

- 1. Fit the model.**
- 2. Display the coefficient and intercept.**

3. **Predict Sales for an advertising expenditure of 60.**
4. **Display actual and predicted values.**

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Create the dataset
data = {
    'Advertising Expenditure': [10, 15, 20, 25, 30, 35, 40, 45, 50],
    'Sales': [25, 32, 38, 45, 51, 58, 64, 70, 77]
}

df = pd.DataFrame(data)

# Independent and dependent variables
X = df[['Advertising Expenditure']]
y = df['Sales']

# Create and fit model
model = LinearRegression()
model.fit(X, y)

# Display coefficient and intercept
print( "Coefficient:", model.coef_[0])
print( "Intercept:", model.intercept_)

# Predict Sales for expenditure = 60
new_data = pd.DataFrame(
    [[60]],
    columns =['Advertising Expenditure']
)

prediction = model.predict(new_data)

print( "\nPredicted Sales for expenditure 60:", prediction[0])

# Predict values for existing data
df['Predicted Sales'] = model.predict(X)
```

```
print( "\nActual and Predicted Values:")
print( df)
```

Q3. Viva [5 Marks]

SLIP 4

Q1. [10 Marks]

Create a small dataset containing monthly sales for six months . Using Matplotlib, create a line plot with a suitable title, X-axis label, Y-axis label and markers.

Write one observation from the graph.

```
import matplotlib.pyplot as plt

# Create monthly sales data
months = ['January', 'February', 'March', 'April', 'May', 'June']
sales = [12000, 15000, 13500, 18000, 21000, 24000]

# Create line plot
plt.plot(months, sales, marker='o')

# Add title and labels
plt.title('Monthly Sales')
plt.xlabel('Month')
plt.ylabel('Sales')

# Display the graph
plt.show()
```

Q2. [20 Marks]

Using the Iris dataset , perform correlation analysis on its numerical features.

- 1. Load the dataset.**
- 2. Select the numerical features.**
- 3. Calculate the correlation matrix.**
- 4. Display the correlation matrix using a heatmap with annotations.**
- 5. Identify and interpret one strong positive or negative relationship.**

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# Load the Iris dataset
df = pd.read_csv("Iris.csv")

# Display first 5 records
print( "First 5 records:")
print( df.head())

# Select numerical features
numeric_data = df.select_ dtypes(include='number')

print( "\nNumerical Features:")
print( numeric_data.head())

# Calculate correlation matrix
correlation_matrix = numeric_ data.corr()

print( "\nCorrelation Matrix:")
print( correlation_matrix)

# Display correlation matrix using heatmap
plt.figure( figsize=(8, 6))
sns.heatmap( correlation_matrix, annot=True, cmap='coolwarm')

plt.title( 'Correlation Matrix of Iris Dataset')
plt.show()
```

Q3. Viva [5 Marks]

SLIP 5

Q1. [10 Marks]

Create a small DataFrame containing five products and their prices, where the price values are initially stored as strings. Convert the price column into an appropriate numerical datatype and display the DataFrame with its data types.

```
import pandas as pd

# Create DataFrame with prices stored as strings
data = {
    'Product': ['Laptop', 'Mobile', 'Headphones', 'Keyboard', 'Mouse'],
    'Price': ['55000', '25000', '2000', '1500', '800']
}

df = pd.DataFrame(data)

print( "Original DataFrame:")
print( df)

print( "\nOriginal Data Types:")
```

```
print( df.dtypes)
```

```
# Convert Price column to numerical datatype  
df['Price'] = pd.to_numeric(df['Price'])
```

```
print( "\nDataFrame after conversion:")  
print( df)
```

```
print( "\nData Types after conversion:")  
print( df.dtypes)
```

Q2. [20 Marks]

Using a suitable dataset, perform Exploratory Data Analysis (EDA) .

- 1. Display the first five records.**
- 2. Display the dimensions and column names.**
- 3. Display data types and statistical summary.**
- 4. Identify and count missing values.**
- 5. Create a suitable visualization to show an important pattern or relationship.**
- 6. Write two observations based on the analysis.**

```
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns
```

```
# Load the dataset  
df = pd.read_csv("Iris.csv")
```

```
# 1. Display first five records  
print( "First Five Records:")  
print( df.head())
```

```
# 2. Display dimensions and column names  
print( "\nDimensions of Dataset:")  
print( df.shape)
```

```
print( "\nColumn Names:")  
print( df.columns)
```

```
# 3. Display data types and statistical summary
```

```

print( "\nData Types:")
print( df.dtypes)

print( "\nStatistical Summary:")
print( df.describe())

# 4. Identify and count missing values
print( "\nMissing Values:")
print( df.isnull().sum())

# 5. Visualization - Scatter plot
plt.figure( figsize=(8, 6))

sns.scatterplot(
    data=df,
    x='PetalLengthCm',
    y='PetalWidthCm',
    hue='Species'
)

plt.title( 'Petal Length vs Petal Width')
plt.xlabel( 'Petal Length')
plt.ylabel( 'Petal Width')
plt.show()

```

Q3. Viva [5 Marks]

SLIP 6

Q1. [10 Marks]

Create a Pandas DataFrame containing the names, ages and marks of five students .
Display the first and last five records, the dimensions of the DataFrame, and the

data types of its columns .

```
import pandas as pd

# Create DataFrame
data = {
    'Name': ['Aarav', 'Priya', 'Rahul', 'Sneha', 'Ananya'],
    'Age': [20, 21, 19, 22, 20],
    'Marks': [75, 88, 65, 92, 78]
}

df = pd.DataFrame(data)

# Display first five records
print( "First Five Records:")
print( df.head())

# Display last five records
print( "\nLast Five Records:")
print( df.tail())

# Display dimensions
print( "\nDimensions of DataFrame:")
print( df.shape)

# Display data types
print( "\nData Types:")
print( df.dtypes)
```

Q2. [20 Marks]

Using the House Price dataset , build a Multiple Linear Regression model using Area, Bedrooms and Age as independent variables and Price as the dependent variable.

- 1. Fit the model.**
- 2. Display coefficients and intercept.**
- 3. Evaluate the model using R^2 , MAE, MSE and RMSE.**
- 4. Display actual and predicted values.**

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
```

```

from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error,
mean_squared_error

# Load dataset
df = pd.read_csv("housing.csv")

# Remove missing values
df = df.dropna()

# Independent variables
X = df[['total_rooms', 'total_bedrooms', 'housing_median_age']]

# Dependent variable
y = df['median_house_value']

# Split dataset
X_train, X_test, y_train, y_test = train_test_      split(
    X, y, test_size=0.2, random_state=42
)

# 1. Fit the model
model = LinearRegression()
model.fit(X_train, y_train)

# 2. Display coefficients and intercept
print("Coefficients:")
print("Total Rooms:", model.coef_[0])
print("Total Bedrooms:", model.coef_[1])
print("Housing Median Age:", model.coef_[2])

print("\nIntercept:", model.intercept_)

# Predict test data
y_pred = model.predict(X_test)

# 3. Evaluate the model
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)

```

```
rmse = np.sqrt(mse)
```

```
print( "\nModel Evaluation:")
```

```
print( "R2 Score:", r2)
```

```
print( "MAE:", mae)
```

```
print( "MSE:", mse)
```

```
print( "RMSE:", rmse)
```

```
# 4. Display actual and predicted values
```

```
result = pd.DataFrame({  
    'Actual Price': y_test.values,  
    'Predicted Price': y_pred  
})
```

```
print( "\nActual and Predicted Values:")
```

```
print( result)
```

Q3. Viva [5 Marks]

SLIP 7

Q1. [10 Marks]

Create a small DataFrame containing five students and their marks . Create a bar chart to compare their marks. Add a suitable title and axis labels and write one observation .

```
import pandas as pd
import matplotlib.pyplot as plt

# Create DataFrame
data = {
    'Student': ['Aarav', 'Priya', 'Rahul', 'Sneha', 'Ananya'],
    'Marks': [75, 88, 65, 92, 78]
}

df = pd.DataFrame(data)

print( "Student Data:")
print( df)

# Create bar chart
plt.bar( df['Student'], df['Marks'])
```

```
plt.title( 'Student Marks')
plt.xlabel( 'Students')
plt.ylabel( 'Marks')
```

```
plt.show()
```

Q2. [20 Marks]

Using the Student Scores , perform K-Means Clustering using Hours and Scores as features.

1. Apply K-Means using the K= 3.
2. Display cluster labels and centroids.
3. Visualize the clusters using a scatter plot.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Load dataset
df = pd.read_csv("student_scores.csv")

# Select features
X = df[['Hours', 'Scores']]

# Apply K-Means with K = 3
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

df['Cluster'] = kmeans.fit_predict(X)

# Display cluster labels
print( "Cluster Labels:")
print( df[['Hours', 'Scores', 'Cluster']])

# Display cluster centroids
print( "\nCluster Centroids:")
print( kmeans.cluster_centers_)

# Visualize clusters
plt.scatter(
    df['Hours'],
    df['Scores'],
```

```

        c=df['Cluster']
    )

    # Display centroids
    plt.scatter(
        kmeans.cluster_centers_[:, 0],
        kmeans.cluster_centers_[:, 1],
        marker='X',
        s=200,
        label='Centroids'
    )

    plt.title("K-Means Clustering of Student Scores")
    plt.xlabel("Hours")
    plt.ylabel("Scores")
    plt.legend()
    plt.show()

```

Q3. Viva [5 Marks]

SLIP 8

Q1. [10 Marks]

Create a small DataFrame containing monthly sales for six months . Create a line plot with suitable title, labels and markers. State whether the sales show an increasing or decreasing trend.

```

import pandas as pd
import matplotlib.pyplot as plt

```

```
# Create DataFrame
data = {
    'Month': ['January', 'February', 'March', 'April', 'May', 'June'],
    'Sales': [12000, 14000, 13500, 16000, 18000, 20000]
}

df = pd.DataFrame(data)

print( "Monthly Sales Data:")
print( df)

# Create line plot
plt.plot( df['Month'], df['Sales'], marker='o')

plt.title( 'Monthly Sales Trend')
plt.xlabel( 'Month')
plt.ylabel( 'Sales')
plt.show()
```

Q2. [20 Marks]

Using the any dataset , build a Multiple Linear Regression model.

- 1. Display the regression coefficients.**
- 2. Make predictions.**
- 3. Evaluate the model using R^2 , MAE, MSE and RMSE.**

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

# Load Housing( 1) dataset
df = pd.read_csv("housing.csv")

# Remove missing values
df = df.dropna()

# Select independent and dependent variables
```

```
X = df[['total_rooms', 'total_bedrooms', 'housing_median_age']]
y = df['median_house_value']
```

```
# Split the dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Build and train the model
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
# 1. Display regression coefficients
```

```
print("Regression Coefficients:")
```

```
for feature, coefficient in zip(X.columns, model.coef_):
    print(feature, ":", coefficient)
```

```
print("\nIntercept:", model.intercept_)
```

```
# 2. Make predictions
```

```
y_pred = model.predict(X_test)
```

```
print("\nPredicted Values:")
```

```
print(y_pred)
```

```
# 3. Evaluate the model
```

```
r2 = r2_score(y_test, y_pred)
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
```

```
print("\nModel Evaluation:")
```

```
print("R2 Score:", r2)
```

```
print("MAE:", mae)
```

```
print("MSE:", mse)
```

```
print("RMSE:", rmse)
```

Viva [5 Marks]

SLIP 9

Q1. [10 Marks]

Create a DataFrame containing five products and their prices , with one missing price value. Identify the missing value and replace it with the mean price.

```
import pandas as pd

# Create DataFrame
data = {
    'Product': ['Laptop', 'Mobile', 'Headphones', 'Keyboard', 'Mouse'],
    'Price': [55000, 25000, None, 1500, 800]
}

df = pd.DataFrame(data)

print( "Original DataFrame:")
print( df)

# Identify missing value
print( "\nMissing Values:")
print( df.isnull())

# Calculate mean price
mean_price = df['Price'].mean()
print( "\nMean Price:", mean_price)

# Replace missing value with mean
df['Price'] = df['Price'].fillna(mean_price)

print( "\nDataFrame after replacing missing value:")
print( df)
```

Q2. [20 Marks]

Using the DATA, build a Logistic Regression model to classify handwritten digits.

1. **Load the dataset.**
2. **Split it into training and testing sets.**
3. **Train the Logistic Regression model.**
4. **Predict the test data.**
5. **Calculate accuracy and display the confusion matrix.**
- 6.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
# 1. Load dataset
```

```
df = pd.read_csv("DATA.csv")
```

```
# Display missing values
```

```
print( "Missing Values:")
```

```
print( df.isnull().sum())
```

```
# 2. Handle missing values
```

```
df['Age'] = df['Age'].fillna(df['Age'].mean())
```

```
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
```

```
# Convert Country into numerical values
```

```
df['Country'] = LabelEncoder().fit_transform(df['Country'])
```

```
# Convert Purchased into 0 and 1
```

```
df['Purchased'] = df['Purchased'].map({'No': 0, 'Yes': 1})
```

```
# Remove any remaining rows containing missing values
```

```
df = df.dropna()
```

```
# 3. Separate features and target
```

```
X = df[['Country', 'Age', 'Salary']]
```

```
y = df['Purchased']
```

```
# Check that there are no missing values
```

```
print( "\nMissing values in X:")
```

```
print( X.isnull().sum())
```

```
# 4. Split the dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

```
# 5. Train Logistic Regression
```

```
model = LogisticRegression(max_iter=1000)  
model.fit(X_train, y_train)
```

```
# 6. Predict
```

```
y_pred = model.predict(X_test)
```

```
# 7. Accuracy
```

```
print( "\nAccuracy:", accuracy_score(y_test, y_pred))
```

```
# 8. Confusion Matrix
```

```
print( "\nConfusion Matrix:")  
print( confusion_matrix(y_test, y_pred))
```

Q3. Viva [5 Marks]

SLIP 10

Q1. [10 Marks]

Create a small DataFrame containing five students and their marks . Display the statistical summary and identify the student with the highest marks.

```
import pandas as pd
```

```
# Create DataFrame
```

```
data = {  
    'Student Name': ['Aarav', 'Priya', 'Rahul', 'Sneha', 'Ananya'],  
    'Marks': [75, 88, 65, 92, 78]  
}
```

```
df = pd.DataFrame(data)
```

```
# Display DataFrame
```

```
print( "Student Data:")  
print( df)
```

```
# Display statistical summary
print( "\nStatistical Summary:")
print( df['Marks'].describe())

# Identify student with highest marks
highest = df.loc[df['Marks'].idxmax()]

print( "\nStudent with Highest Marks:")
print( highest)
```

Q2. [20 Marks]

Perform EDA on the Wine Quality dataset .

- 1. Display the first five records.**
- 2. Display dimensions and data types.**
- 3. Display the statistical summary.**
- 4. Identify missing values.**
- 5. Create a suitable visualization to show an important relationship in the dataset.**
- 6. Write two observations.**

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load the dataset
df = pd.read_csv("winequality-red.csv", sep=';')

# 1. Display first five records
print( "First Five Records:")
print( df.head())

# 2. Display dimensions and data types
print( "\nDimensions of Dataset:")
print( df.shape)

print( "\nData Types:")
print( df.dtypes)

# 3. Display statistical summary
print( "\nStatistical Summary:")
```

```
print( df.describe())
```

```
# 4. Identify missing values
```

```
print( "\nMissing Values:")
```

```
print( df.isnull().sum())
```

```
# 5. Visualization: Alcohol vs Quality
```

```
plt.figure( figsize=(8, 6))
```

```
sns.scatterplot( data=df, x='alcohol', y='quality')
```

```
plt.title( "Alcohol Content vs Wine Quality")
```

```
plt.xlabel( "Alcohol")
```

```
plt.ylabel( "Quality")
```

```
plt.show()
```

```
# 6. Display correlation between alcohol and quality
```

```
print( "\nCorrelation between Alcohol and Quality:")
```

```
print( df['alcohol'].corr(df['quality']))
```

Q3. Viva [5 Marks]

SLIP 11

Q1. [10 Marks]

Create a small DataFrame containing five students with their ages stored as strings .

Convert the Age column into integer datatype and display the resulting data types.

```

import pandas as pd

# Create DataFrame
data = {
    'Student Name': ['Aarav', 'Priya', 'Rahul', 'Sneha', 'Ananya'],
    'Age': ['20', '21', '19', '22', '20']
}

df = pd.DataFrame(data)

print( "Original DataFrame:")
print( df)

print( "\nOriginal Data Types:")
print( df.dtypes)

# Convert Age from string to integer
df['Age'] = df['Age'].astype(int)

print( "\nDataFrame after conversion:")
print( df)

print( "\nData Types after conversion:")
print( df.dtypes)

```

Q2. [20 Marks]

Using the given Fish dataset, build a Simple Linear Regression model to predict Weight based on Length1.

1. **Fit the model.**
2. **Display the coefficient and intercept.**
3. **Predict Weight for a fish having Length1 = 60 .**
4. **Calculate predicted values for the dataset.**
5. **Plot the data points and regression line.**

```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Load the Fish dataset
df = pd.read_csv("Fish.csv")

```

```
# Independent and dependent variables
```

```
X = df[['Length1']]
```

```
y = df['Weight']
```

```
# 1. Fit the Simple Linear Regression model
```

```
model = LinearRegression()
```

```
model.fit(X, y)
```

```
# 2. Display coefficient and intercept
```

```
print( "Coefficient:", model.coef_[0])
```

```
print( "Intercept:", model.intercept_)
```

```
# 3. Predict Weight for Length1 = 60
```

```
new_data = pd.DataFrame( [[60]], columns=['Length1'])
```

```
prediction = model.predict(new_data)
```

```
print( "\nPredicted Weight for Length1 = 60:")
```

```
print( prediction[0])
```

```
# 4. Calculate predicted values
```

```
y_pred = model.predict(X)
```

```
print( "\nPredicted Values:")
```

```
print( y_pred)
```

```
# 5. Plot data points and regression line
```

```
plt.scatter(X, y, label='Actual Data')
```

```
plt.plot(X, y_pred, label='Regression Line')
```

```
plt.xlabel( "Length1")
```

```
plt.ylabel( "Weight")
```

```
plt.title( "Length1 vs Weight")
```

```
plt.legend()
```

```
plt.show()
```

Q3. Viva [5 Marks]

SLIP 12

Q1. [10 Marks]

Create a small dataset containing six students and their marks . Display the dataset using a histogram with suitable bins and labels. Write one observation about the distribution.

```
import pandas as pd
import matplotlib.pyplot as plt

# Create DataFrame
data = {
    'Student Name': ['Aarav', 'Priya', 'Rahul', 'Sneha', 'Ananya', 'Rohan'],
    'Marks': [45, 62, 75, 82, 68, 55]
}

df = pd.DataFrame(data)

# Display dataset
print( "Student Data:")
print( df)

# Create histogram
plt.hist( df['Marks'], bins=[40, 50, 60, 70, 80, 90], edgecolor='black')

plt.xlabel( "Marks")
plt.ylabel( "Number of Students")
plt.title( "Distribution of Student Marks")
plt.show()
```

Q2. [20 Marks]

Perform correlation analysis on the Iris dataset .

1. **Load the dataset.**
2. **Select numerical features.**
3. **Calculate the correlation matrix.**

4. **Display it using a heatmap with annotations.**
5. **Identify the strongest positive or negative relationship.**
6. **Interpret the relationship.**

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Load the dataset
df = pd.read_csv("iris.csv")

# 2. Select numerical features
numeric_data = df.select_dtypes(include='number')

# Remove Id column if present
numeric_data = numeric_data.drop(columns=['Id'], errors='ignore')

print("Numerical Features:")
print(numeric_data.head())

# 3. Calculate correlation matrix
correlation_matrix = numeric_data.corr()

print("\nCorrelation Matrix:")
print(correlation_matrix)

# 4. Display correlation matrix using heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f')

plt.title("Correlation Matrix of Iris Dataset")
plt.show()

# 5. Find strongest relationship
corr = correlation_matrix.copy()

# Remove diagonal values
for i in range(len(corr)):
    corr.iloc[i, i] = 0
```

```
strongest_pair = corr.abs().stack().idxmax()
strongest_value = corr.loc[strongest_pair]

print( "\nStrongest Relationship:")
print( strongest_pair[0], "and", strongest_pair[1])
print( "Correlation:", strongest_value)
```

Q3. Viva [5 Marks]

SLIP 13

Q1. [10 Marks]

Create a small DataFrame containing five students and their marks . Display students scoring more than 70 marks and sort them in descending order of marks.

```
import pandas as pd

# Create DataFrame
data = {
    'Student Name': ['Aarav', 'Priya', 'Rahul', 'Sneha', 'Ananya'],
    'Marks': [75, 55, 88, 62, 48]
}

df = pd.DataFrame(data)

print( "Original DataFrame:")
print( df)

# Select students scoring more than 70
result = df[df['Marks'] > 70]
```

```
# Sort in descending order of marks
result = result.sort_values(by='Marks', ascending=False)

print( "\nStudents scoring more than 70 marks:")
print( result)
```

Q2. [20 Marks]

Using the Housing(1) dataset, perform Simple Linear Regression to predict MedHouseValue using a suitable numerical feature.

1. **Select the independent and dependent variables.**
2. **Split the data into training and testing sets.**
3. **Fit the regression model.**
4. **Display coefficient and intercept.**
5. **Perform residual analysis using a suitable plot.**

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```
# 1. Load the dataset
df = pd.read_csv("Housing(1).csv")
```

```
# 2. Select independent and dependent variables
X = df[['area']]
y = df['price']
```

```
# 3. Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# 4. Fit the regression model
model = LinearRegression()
model.fit( X_train, y_train)
```

```
# 5. Display coefficient and intercept
print( "Coefficient:", model.coef_[0])
print( "Intercept:", model.intercept_)
```

```
# Predict test data
y_pred = model.predict( X_test)

# 6. Residual analysis
residuals = y_test - y_pred

plt.figure( figsize=(8, 6))
plt.scatter( y_pred, residuals)

plt.axhline(y=0, linestyle='--')

plt.xlabel( "Predicted Price")
plt.ylabel( "Residuals")
plt.title( "Residual Plot")
plt.show()
```

Q3. Viva [5 Marks]

SLIP 14

Q1. [10 Marks]

Create a small dataset containing six months of temperature and rainfall values .
Create a scatter plot showing the relationship between temperature and rainfall and provide one observation.

```

import pandas as pd
import matplotlib.pyplot as plt

# Create dataset
data = {
    'Month': ['January', 'February', 'March', 'April', 'May', 'June'],
    'Temperature': [22, 24, 28, 32, 35, 30],
    'Rainfall': [10, 15, 20, 45, 80, 120]
}

df = pd.DataFrame(data)

print( "Temperature and Rainfall Data:")
print( df)

# Create scatter plot
plt.scatter( df['Temperature'], df['Rainfall'])

plt.xlabel( "Temperature (°C)")
plt.ylabel( "Rainfall (mm)")
plt.title( "Temperature vs Rainfall")
plt.show()

```

Q2. [20 Marks]

Using the Customers, perform K-Means Clustering.

1. **Select two suitable numerical features.**
2. **Apply K-Means clustering.**
3. **Display cluster labels and centroids.**
4. **Visualize the clusters.**
5. **Interpret the clusters.**

```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# 1. Load the dataset
df = pd.read_csv("Customers.csv")

# 2. Select two numerical features
X = df[['Annual_Income', 'Spending_Score']]

```

```
# 3. Apply K-Means Clustering
```

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
```

```
df['Cluster'] = kmeans.fit_predict(X)
```

```
# 4. Display cluster labels
```

```
print( "Cluster Labels:")
```

```
print( df[['CustomerID', 'Annual_Income', 'Spending_Score', 'Cluster']])
```

```
# Display centroids
```

```
print( "\nCluster Centroids:")
```

```
print( kmeans.cluster_centers_)
```

```
# 5. Visualize the clusters
```

```
plt.figure( figsize=(8, 6))
```

```
plt.scatter(
```

```
    df['Annual_Income'],
```

```
    df['Spending_Score'],
```

```
    c=df['Cluster']
```

```
)
```

```
plt.scatter(
```

```
    kmeans.cluster_centers_[:, 0],
```

```
    kmeans.cluster_centers_[:, 1],
```

```
    marker ='X',
```

```
    s=200,
```

```
    label ='Centroids'
```

```
)
```

```
plt.xlabel( "Annual Income")
```

```
plt.ylabel( "Spending Score")
```

```
plt.title( "Customer Segmentation using K-Means")
```

```
plt.legend()
```

```
plt.show()
```

Q3. Viva [5 Marks]

SLIP 15

Q1. [10 Marks]

Create a small DataFrame containing five categories and their values . Display the values using a pie chart with suitable labels and percentages. Write one observation.

```
import pandas as pd
import matplotlib.pyplot as plt

# Create DataFrame
data = {
    'Category': ['Food', 'Travel', 'Shopping', 'Education', 'Entertainment'],
    'Value': [30, 20, 25, 15, 10]
}

df = pd.DataFrame(data)

print( "DataFrame:")
print( df)

# Create pie chart
plt.pie(
    df['Value'],
    labels= df['Category'],
    autopct = '%1.1f%%',
    startangle= 90
)

plt.title( "Category-wise Values")
plt.show()
```

Q2. [20 Marks]

Using the Cancer dataset , build a Logistic Regression classification model.

1. **Load the dataset.**
2. **Split the data into training and testing sets.**

3. **Train the model.**
4. **Predict the test observations.**
5. **Calculate accuracy.**

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 1. Load the dataset
df = pd.read_csv("cancer.csv")

# Remove unnecessary columns
df = df.drop(['id', 'Unnamed: 32'], axis=1, errors='ignore')

# Convert diagnosis into numerical values
df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})

# Select features and target
X = df.drop('diagnosis', axis=1)
y = df['diagnosis']

# 2. Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Train the Logistic Regression model
model = LogisticRegression(max_iter=10000)
model.fit(X_train, y_train)

# 4. Predict the test observations
y_pred = model.predict(X_test)

print("Predicted Values:")
print(y_pred)

# 5. Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
```

```
print( "\nAccuracy:", accuracy)
```

Q3. Viva [5 Marks]

SLIP 16

Q1. [10 Marks]

Create a small DataFrame containing five employees and their salaries . Display the statistical summary and create a suitable boxplot of salaries. State one observation.

```
import pandas as pd
import matplotlib.pyplot as plt

# Create DataFrame
data = {
    'Employee': ['Aarav', 'Priya', 'Rahul', 'Sneha', 'Ananya'],
    'Salary': [30000, 45000, 38000, 52000, 40000]
}

df = pd.DataFrame(data)

# Display DataFrame
print( "Employee Data:")
```

```
print( df)
```

```
# Statistical summary
print( "\nStatistical Summary:")
print( df['Salary'].describe())
```

```
# Create boxplot
plt.boxplot( df['Salary'])
```

```
plt.ylabel( "Salary")
plt.title( "Salary Distribution")
plt.show()
```

Q2. [20 Marks]

Using the given Housing , perform Multiple Linear Regression using Area, Bedrooms to predict Price.

1. **Split the dataset into training and testing sets.**
2. **Fit the model.**
3. **Display coefficients and intercept.**
4. **Predict prices for test data.**
5. **Evaluate the model using R^2 , MAE, MSE and RMSE.**

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error,
mean_squared_error
import numpy as np
```

```
# Load the dataset
df = pd.read_csv("Housing(1).csv")
```

```
# Select independent and dependent variables
X = df[['area', 'bedrooms', 'bathrooms']]
y = df['price']
```

```
# 1. Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

2. Fit the Multiple Linear Regression model

```
model = LinearRegression()
```

```
model.fit( X_train, y_train)
```

3. Display coefficients and intercept

```
print( "Coefficients:")
```

```
print( "Area:", model.coef_[0])
```

```
print( "Bedrooms:", model.coef_[1])
```

```
print( "Age:", model.coef_[2])
```

```
print( "\nIntercept:", model.intercept_)
```

4. Predict prices for test data

```
y_pred = model.predict( X_test)
```

```
print( "\nPredicted Prices:")
```

```
print( y_pred)
```

5. Evaluate the model

```
r2 = r2_score(y_test, y_pred)
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
```

```
print( "\nModel Evaluation:")
```

```
print( "R²:", r2)
```

```
print( "MAE:", mae)
```

```
print( "MSE:", mse)
```

```
print( "RMSE:", rmse)
```

Q3. Viva [5 Marks]

SLIP 17

Q1. [10 Marks]

Create a small DataFrame containing five products and their sales . Display the products with sales above a suitable value and create a bar chart of the filtered

data.

```
import pandas as pd
import matplotlib.pyplot as plt

# Create DataFrame
data = {
    'Product': ['Laptop', 'Mobile', 'Headphones', 'Keyboard', 'Mouse'],
    'Sales': [80, 120, 60, 95, 45]
}

df = pd.DataFrame(data)

print( "Original DataFrame:")
print( df)

# Filter products with sales above 70
filtered_df = df[df['Sales'] > 70]

print( "\nProducts with sales above 70:")
print( filtered_df)

# Create bar chart
plt.bar( filtered_df['Product'], filtered_df['Sales'])

plt.xlabel( "Product")
plt.ylabel( "Sales")
plt.title( "Products with Sales Above 70")
plt.show()
```

Q2. [20 Marks]

Using the Diabetes dataset , perform Simple Linear Regression using BMI as the independent variable and outcome as the dependent variable.

- 1. Split the data.**
- 2. Train the model.**
- 3. Display coefficient and intercept.**
- 4. Predict test values.**
- 5. Calculate residuals.**
- 6. Visualize the regression line.**

```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Load dataset
df = pd.read_csv("diabetes.csv")

# Independent and dependent variables
X = df[['BMI']]
y = df['Outcome']

# 1. Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 2. Train the model
model = LinearRegression()
model.fit(X_train, y_train)

# 3. Display coefficient and intercept
print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)

# 4. Predict test values
y_pred = model.predict(X_test)

print("\nPredicted Test Values:")
print(y_pred)

# 5. Calculate residuals
residuals = y_test - y_pred

print("\nResiduals:")
print(residuals)

# 6. Visualize the regression line
plt.scatter(X_test, y_test, label="Actual Data")
plt.plot(X_test, y_pred, label="Regression Line")

```

```
plt.xlabel( "BMI")
plt.ylabel( "Outcome")
plt.title( "Simple Linear Regression: BMI vs Outcome")
plt.legend()
plt.show()
```

Q3. Viva [5 Marks]

SLIP 18

Q1. [10 Marks]

Create a DataFrame containing five students and their marks , including two missing values. Identify the missing values and replace them using an appropriate method. Display the cleaned DataFrame.

```
import pandas as pd

# Create DataFrame
data = {
    'Student Name': ['Aarav', 'Priya', 'Rahul', 'Sneha', 'Ananya'],
    'Marks': [75, None, 88, None, 65]
}

df = pd.DataFrame(data)

# Display original DataFrame
print( "Original DataFrame:")
print( df)

# Identify missing values
print( "\nMissing Values:")
print( df.isnull())

# Calculate mean of available marks
mean_marks = df['Marks'].mean()

print( "\nMean Marks:", mean_marks)

# Replace missing values with mean
```

```
df['Marks'] = df['Marks'].fillna(mean_marks)
```

```
# Display cleaned DataFrame  
print( "\nCleaned DataFrame:")  
print( df)
```

Q2. [20 Marks]

Using the HR Data dataset , perform Logistic Regression classification.

1. **Load and inspect the dataset.**
2. **Split the data into training and testing sets.**
3. **Train the classifier.**
4. **Predict the test observations.**
5. **Calculate accuracy.**
6. **Display the confusion matrix.**

```
import pandas as pd  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import LabelEncoder  
from sklearn.linear_model import LogisticRegression  
from sklearn.metrics import accuracy_score, confusion_matrix
```

```
# 1. Load and inspect the dataset  
df = pd.read_excel("HR_Data.xlsx ")
```

```
print( "First Five Records:")  
print( df.head())
```

```
print( "\nDataset Shape:")  
print( df.shape)
```

```
print( "\nMissing Values:")  
print( df.isnull().sum())
```

```
# Convert categorical columns into numerical values  
le = LabelEncoder()
```

```
for column in df.select_dtypes(include='object').columns:  
    df[column] = le.fit_transform(df[column])
```

```
# Define input and target
X = df.drop('Attrition', axis=1)
y = df['Attrition']

# 2. Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Train the classifier
model = LogisticRegression(max_iter=5000)
model.fit(X_train, y_train)

# 4. Predict the test observations
y_pred = model.predict(X_test)

print("\nPredicted Test Observations:")
print(y_pred)

# 5. Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:", accuracy)

# 6. Display confusion matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)
```

Q3. Viva [5 Marks]

SLIP 19

Q1. [10 Marks]

Create a small dataset containing six students with study hours and marks . Create a scatter plot showing the relationship between study hours and marks and write one observation.

```
import pandas as pd
import matplotlib.pyplot as plt

# Create dataset
data = {
    'Student': ['Aarav', 'Priya', 'Rahul', 'Sneha', 'Ananya', 'Rohan'],
    'Study Hours': [2, 3, 4, 5, 6, 7],
```

```
'Marks': [45, 52, 60, 68, 75, 82]
}

df = pd.DataFrame(data)

print( "Student Data:")
print( df)

# Create scatter plot
plt.scatter( df['Study Hours'], df['Marks'])

plt.xlabel( "Study Hours")
plt.ylabel( "Marks")
plt.title( "Study Hours vs Marks")
plt.show()
```

Q2. [20 Marks]

Perform EDA on a suitable dataset.

- 1. Display the first five records.**
- 2. Display dimensions and column names.**
- 3. Display data types and statistical summary.**
- 4. Identify missing values.**
- 5. Create two suitable visualizations.**
- 6. Write two meaningful observations from the analysis.**

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
df = pd.read_csv("iris.csv")

# 1. Display first five records
print( "First Five Records:")
print( df.head())

# 2. Display dimensions and column names
print( "\nDimensions of Dataset:")
print( df.shape)
```

```
print( "\nColumn Names:")
print( df.columns)
```

3. Display data types and statistical summary

```
print( "\nData Types:")
print( df.dtypes)
```

```
print( "\nStatistical Summary:")
print( df.describe())
```

4. Identify missing values

```
print( "\nMissing Values:")
print( df.isnull().sum())
```

5. Create two visualizations

Visualization 1: Scatter Plot

```
plt.figure( figsize=(8, 6))
sns.scatterplot(
    data=df,
    x='PetalLengthCm',
    y='PetalWidthCm',
    hue='Species'
)
plt.title( "Petal Length vs Petal Width")
plt.xlabel( "Petal Length")
plt.ylabel( "Petal Width")
plt.show()
```

Visualization 2: Histogram

```
plt.figure( figsize=(8, 6))
plt.hist( df['SepalLengthCm'], bins=10, edgecolor='black')
plt.xlabel( "Sepal Length")
plt.ylabel( "Frequency")
plt.title( "Distribution of Sepal Length")
plt.show()
```

Q3. Viva [5 Marks]

SLIP 20

Q1. [10 Marks]

Create a small DataFrame containing five students, their ages and marks . Display the first five records, column names and dimensions of the DataFrame.

```
import pandas as pd

# Create DataFrame
data = {
    'Student Name': ['Aarav', 'Priya', 'Rahul', 'Sneha', 'Ananya'],
    'Age': [20, 21, 19, 22, 20],
    'Marks': [75, 88, 65, 92, 78]
}

df = pd.DataFrame(data)

# Display first five records
print( "First Five Records:")
print( df.head())

# Display column names
print( "\nColumn Names:")
print( df.columns)

# Display dimensions
print( "\nDimensions of DataFrame:")
print( df.shape)
```

Q2. [20 Marks]

Using the Housing (1) dataset, build a Multiple Linear Regression model. Prepare the independent and dependent variables.

1. **Fit the model.**
2. **Display coefficients.**
3. **Make predictions.**
4. **Evaluate using R^2 , MAE, MSE and RMSE.**

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
```

```
# Load the dataset
df = pd.read_csv("Housing(1).csv")

# Prepare independent and dependent variables
X = df[['area', 'bedrooms', 'bathrooms']]
y = df['price']

# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 1. Fit the Multiple Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# 2. Display coefficients
print("Coefficients:")

print("Area:", model.coef_[0])
print("Bedrooms:", model.coef_[1])
print("Bathrooms:", model.coef_[2])

print("\nIntercept:", model.intercept_)

# 3. Make predictions
y_pred = model.predict(X_test)

print("\nPredicted Prices:")
print(y_pred)

# 4. Evaluate the model
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

print("\nModel Evaluation:")
print("R²:", r2)
```

```
print( "MAE:", mae)
print( "MSE:", mse)
print( "RMSE:", rmse)
```

Q3. Viva [5 Marks]

SLIP 21

Q1. [10 Marks]

Create a small dataset containing monthly expenditure for five categories **. Create a bar chart to compare the expenditures and write one observation.**

```
import pandas as pd
import matplotlib.pyplot as plt

# Create dataset
data = {
    'Category': ['Food', 'Travel', 'Shopping', 'Education', 'Entertainment'],
    'Expenditure': [5000, 3000, 4000, 2500, 2000]
}

df = pd.DataFrame(data)

print( "Monthly Expenditure:")
print( df)

# Create bar chart
plt.bar( df['Category'], df['Expenditure'])

plt.xlabel( "Category")
plt.ylabel( "Monthly Expenditure (₹)")
plt.title( "Monthly Expenditure by Category")
plt.show()
```

Q2. [20 Marks]

Using the Customer dataset , perform Elbow Method to find out the appropriate K

1. **Prepare the data.**
2. **Apply the Elbow Method.**
3. **Select a suitable K.**
4. **Display cluster labels and centroids.**
5. **Visualize and interpret the clusters.**

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

```
# 1. Load the dataset
```

```
df = pd.read_csv("Customers.csv")
```

```
# 2. Select two numerical features
```

```
X = df[['Annual_Income', 'Spending_Score']]
```

```
# 3. Apply Elbow Method
```

```
inertia = []
```

```
for k in range(1, 11):
```

```
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
```

```
    kmeans.fit(X)
```

```
    inertia.append( kmeans.inertia_)
```

```
plt.plot( range(1, 11), inertia, marker='o')
```

```
plt.xlabel( "Number of Clusters (K)")
```

```
plt.ylabel( "Inertia")
```

```
plt.title( "Elbow Method")
```

```
plt.show()
```

```
# 4. Select suitable K
```

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
```

```
df['Cluster'] = kmeans.fit_predict(X)
```

```
# 5. Display cluster labels
```

```
print( "Cluster Labels:")
```

```
print( df[['CustomerID', 'Annual_Income', 'Spending_Score', 'Cluster']])
```

```

# Display centroids
print( "\nCluster Centroids:")
print( kmeans.cluster_centers_)

# 6. Visualize the clusters
plt.figure( figsize=(8, 6))

plt.scatter(
    df['Annual_Income'],
    df['Spending_Score'],
    c=df['Cluster']
)

plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    marker ='X',
    s=200,
    label ='Centroids'
)

plt.xlabel( "Annual Income")
plt.ylabel( "Spending Score")
plt.title( "Customer Segmentation using K-Means")
plt.legend()
plt.show()

```

Q3. Viva [5 Marks]

SLIP 22

Q1. [10 Marks]

Create a small Data Frame containing five students and their marks . Calculate and display the mean, maximum and minimum marks. Create a simple visualization of the marks.

```

import pandas as pd
import matplotlib.pyplot as plt

# Create DataFrame
data = {

```

```
'Student Name': ['Aarav', 'Priya', 'Rahul', 'Sneha', 'Ananya'],  
'Marks': [75, 88, 65, 92, 78]  
}
```

```
df = pd.DataFrame(data)
```

```
# Display DataFrame  
print( "Student Data:")  
print( df)
```

```
# Calculate mean, maximum and minimum  
print( "\nMean Marks:", df['Marks'].mean())  
print( "Maximum Marks:", df['Marks'].max())  
print( "Minimum Marks:", df['Marks'].min())
```

```
# Create visualization  
plt.bar( df['Student Name'], df['Marks'])
```

```
plt.xlabel( "Student Name")  
plt.ylabel( "Marks")  
plt.title( "Student Marks")  
plt.show()
```

Q2. [20 Marks]

Using a suitable numerical dataset, implement Simple Linear Regression .

1. **Select two numerical variables.**
2. **Split the data into training and testing sets.**
3. **Fit the model.**
4. **Display the coefficient and intercept.**
5. **Predict the test values.**
6. **Create a regression-line plot and residual plot.**
7. **Give a brief interpretation.**

```
import pandas as pd  
import matplotlib.pyplot as plt  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression
```

```
# 1. Load the dataset  
df = pd.read_csv("Fish.csv")
```

```

# Select two numerical variables
X = df[['Length1']]
y = df['Weight' ]
# 2. Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# 3. Fit the model
model = LinearRegression()
model.fit( X_train, y_train)

# 4. Display coefficient and intercept
print( "Coefficient:", model.coef_[0])
print( "Intercept:", model.intercept_)

# 5. Predict the test values
y_pred = model.predict( X_test)

print( "\nPredicted Test Values:")
print( y_pred)

# 6. Regression-line plot
plt.figure( figsize=(8, 6))
plt.scatter( X_test, y_test, label="Actual Data")
plt.plot( X_test, y_pred, label="Regression Line")

plt.xlabel( "Length1")
plt.ylabel( "Weight")
plt.title( "Simple Linear Regression")
plt.legend()
plt.show()

# Residual plot
residuals = y_test - y_pred

plt.figure( figsize=(8, 6))
plt.scatter( y_pred, residuals)
plt.axhline(y=0, linestyle='--')

```

```
plt.xlabel( "Predicted Weight")
plt.ylabel( "Residuals")
plt.title( "Residual Plot")
plt.show()
```

Q3. Viva [5 Marks]

SLIP 23

Q1. [10 Marks]

Create a small Data Frame containing five records with a categorical column and a numerical column . **Display the frequency/count of each category and represent it using a bar chart.**

```
import pandas as pd
import matplotlib.pyplot as plt

# Create DataFrame
data = {
    'Department': ['Data Science', 'Computer Science', 'Data Science', 'Computer
Science', 'Data Science'],
    'Marks': [85, 72, 90, 68, 78]
}

df = pd.DataFrame(data)

print( "DataFrame:")
print( df)

# Calculate frequency of each category
frequency = df['Department'].value_counts()

print( "\nFrequency of Each Category:")
print( frequency)

# Create bar chart
plt.bar( frequency.index, frequency.values)

plt.xlabel( "Department")
plt.ylabel( "Frequency")
plt.title( "Frequency of Students by Department")
```

```
plt.show()
```

Q2. [20 Marks]

Perform EDA and correlation analysis on a suitable numerical dataset.

1. **Display basic information and statistical summary.**
2. **Check missing values.**
3. **Calculate the correlation matrix.**
4. **Display the correlation matrix using a heatmap.**
5. **Create one additional suitable visualization.**
6. **Interpret two important observations.**

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Load the dataset
```

```
df = pd.read_csv("iris.csv")
```

```
# 1. Display basic information and statistical summary
```

```
print( "Basic Information:")
```

```
print( df.info())
```

```
print( "\nStatistical Summary:")
```

```
print( df.describe())
```

```
# 2. Check missing values
```

```
print( "\nMissing Values:")
```

```
print( df.isnull().sum())
```

```
# Select numerical columns
```

```
numeric_data = df.select_ dtypes( include='number')
```

```
# Remove Id column if present
```

```
numeric_data = numeric_ data.drop( columns=['Id'], errors='ignore')
```

```
# 3. Calculate correlation matrix
```

```
correlation_matrix = numeric_ data.corr()
```

```
print( "\nCorrelation Matrix:")
```

```
print( correlation_matrix)
```

4. Display correlation matrix using heatmap

```
plt.figure( figsize=(8, 6))
```

```
sns.heatmap(  
    correlation_matrix,  
    annot= True,  
    cmap='coolwarm',  
    fmt='.2f'  
)
```

```
plt.title( "Correlation Matrix Heatmap")
```

```
plt.show()
```

5. Additional visualization

```
plt.figure( figsize=(8, 6))
```

```
sns.scatterplot(  
    data=df,  
    x='PetalLengthCm',  
    y='PetalWidthCm',  
    hue='Species'  
)
```

```
plt.title( "Petal Length vs Petal Width")
```

```
plt.xlabel( "Petal Length")
```

```
plt.ylabel( "Petal Width")
```

```
plt.show()
```

Q3. Viva [5 Marks]

SLIP 24

Q1. [10 Marks]

Create a small Data Frame containing five employees with age and salary , with the Age values initially stored as strings. Convert Age into integer datatype and display the resulting data types.

```
import pandas as pd
```

```
# Create DataFrame
```

```
data = {  
    'Employee': ['Aarav', 'Priya', 'Rahul', 'Sneha', 'Ananya'],  
    'Age': ['25', '30', '28', '35', '27'],  
    'Salary': [30000, 45000, 38000, 52000, 40000]}
```

```

}

df = pd.DataFrame(data)

print( "Original DataFrame:")
print( df)

print( "\nOriginal Data Types:")
print( df.dtypes)

# Convert Age from string to integer
df['Age'] = df['Age'].astype(int)

print( "\nDataFrame after converting Age:")
print( df)

print( "\nResulting Data Types:")
print( df.dtypes)

```

Q2. [20 Marks]

Create the below dataset to perform Simple Linear Regression.

Study Hours	Exam Marks
2	35
3	42
4	48
5	55
6	61
7	67
8	72
9	78

10	84
----	----

1. **Create the dataset using Python.**
2. **Split the data into training and testing sets.**
3. **Fit the Simple Linear Regression model.**
4. **Display the coefficient and intercept.**
5. **Predict the dependent variable for test data.**
6. **Calculate R^2 , MAE, MSE and RMSE .**

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

# 1. Create the dataset
data = {
    'Study Hours': [2, 3, 4, 5, 6, 7, 8, 9, 10, 11],
    'Exam Marks': [35, 42, 48, 55, 61, 67, 72, 78, 84, 89]
}

df = pd.DataFrame(data)

print( "Dataset:")
print( df)

# Independent and dependent variables
X = df[['Study Hours']]
y = df['Exam Marks']

# 2. Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Fit the Simple Linear Regression model
model = LinearRegression()
model.fit( X_train, y_train)
```

```
# 4. Display coefficient and intercept
print( "\nCoefficient:", model.coef_[0])
print( "Intercept:", model.intercept_)
```

```
# 5. Predict the dependent variable for test data
y_pred = model.predict( X_test)
```

```
print( "\nPredicted Exam Marks:")
print( y_pred)
```

```
# 6. Calculate evaluation metrics
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
```

```
print( "\nModel Evaluation:")
print( "R²:", r2)
print( "MAE:", mae)
print( "MSE:", mse)
print( "RMSE:", rmse)
```

Q3. Viva [5 Marks]

SLIP 25

Q1. [10 Marks]

Create a small DataFrame containing six products and their prices , including one duplicate record. Identify and remove the duplicate record. Display the cleaned DataFrame and its dimensions.

```
import pandas as pd

# Create DataFrame with one duplicate record
data = {
    'Product': ['Laptop', 'Mobile', 'Headphones', 'Keyboard', 'Mouse', 'Mobile'],
    'Price': [55000, 25000, 2000, 1500, 800, 25000]
}

df = pd.DataFrame(data)

print( "Original DataFrame:")
print( df)

# Remove duplicate records
df = df.drop_duplicates()

print( "\nCleaned DataFrame:")
print( df)

# Display dimensions
print( "\nDimensions of Cleaned DataFrame:")
print( df.shape)
```

Q2. [20 Marks]

Using the Cancer dataset , build and evaluate a Logistic Regression classification model.

1. **Load the dataset.**
2. **Split the data into training and testing sets.**
3. **Train the Logistic Regression model.**
4. **Predict the test observations.**
5. **Calculate accuracy.**

6. Display the confusion matrix and briefly interpret the result.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# 1. Load the dataset
df = pd.read_csv("cancer.csv")

# Remove unnecessary columns
df = df.drop(["id", "Unnamed: 32"], axis=1, errors="ignore")

# Convert diagnosis into numerical values
df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})

# Define input and output
X = df.drop('diagnosis', axis=1)
y = df['diagnosis']

# 2. Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Train the Logistic Regression model
model = LogisticRegression(max_iter=10000)
model.fit(X_train, y_train)

# 4. Predict the test observations
y_pred = model.predict(X_test)

print("Predicted Test Observations:")
print(y_pred)

# 5. Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy:", accuracy)
```

